Dependency-Aware Transaction Scheduling for Database Makespan Minimization

Anonymous Author(s)
Anonymous Institution

Abstract

Modern in-memory database systems face significant challenges when scheduling transactions under high-contention workloads. Existing schedule-first approaches typically rely on greedy local heuristics that easily fall into suboptimal local minima within dense conflict graphs. Conversely, complex scheduling methods can introduce computational overhead that challenges the strict latency constraints of transaction processing. To address these limitations, we propose a pure algorithmic, dependency-aware transaction scheduling framework. Our approach constructs an initial schedule using a robust, randomized beam search enabled by global cost caching, which effectively avoids massive conflicts early in the search process. This prefix-building phase is followed by a multi-neighborhood stochastic hill climber that refines the schedule using swap, relocate, and a novel block-reversal operator to escape local minima while preserving locally optimal sub-sequences. Empirical evaluations demonstrate that our method achieves a highly competitive overall score of 3906.25 and a minimal makespan of 255.0. This performance significantly outperforms standard sequential baselines and prior greedy scheduling algorithms. Ultimately, our findings highlight that a carefully designed stochastic heuristic with diverse sequence mutations can effectively navigate complex transaction dependency graphs, achieving superior makespan minimization without excessive computational burden.

1 Introduction

Main-memory online transaction processing (OLTP) database management systems (DBMS) increasingly rely on multi-core architectures to handle high-throughput workloads. However, as concurrency increases, the probability of transaction conflictsparticularly in high-contention scenariosrises significantly [Zhang et al., 2024]. Traditional concurrency control mechanisms, such as two-phase locking (2PL) or optimistic concurrency control (OCC), often struggle to balance transaction dependency constraints with the need for high concurrency [Chen et al., 2025]. When multiple transactions attempt to access the same data items, reactive protocols inevitably lead to high abort rates and wasted computational cycles. Consequently, researchers have shifted toward intelligent transaction scheduling, which aims to estimate potential conflicts and proactively order transactions to minimize the overall makespan and avoid runtime aborts [Zhang et al., 2024]. By treating transaction execution as a dependency-aware scheduling problem, systems can theoretically achieve optimal throughput.

Despite these conceptual advances, existing scheduling paradigms exhibit critical practical limitations. Schedule-first approaches, such as Shortest Makespan First (SMF), typically rely on greedy local heuristics to construct execution orders [Cheng et al., 2024]. For example, SMF samples a small subset of $k = 5$ transactions and greedily appends the one that minimizes the incremental makespan. While computationally inexpensive, these greedy choices fail to account for the global critical path of the dependency graph. Consequently, they frequently trap the scheduler in suboptimal local minima when navigating dense, highly connected conflict clusters. Alternatively, recent methods explore

Preprint.

intelligent transaction scheduling based on transaction decomposition and dependency constraints to predict conflicts and schedule operations [Chen et al., 2025]. Although these approaches are highly effective at reducing makespan, they can introduce substantial computational overhead. In a high-performance DBMS, scheduling decisions must be made in microseconds; the latency introduced by complex predictive models can challenge the strict timing constraints required by pure algorithmic solvers. Furthermore, conventional non-preemptive scheduling strategies struggle to adapt to mixed workloads, where long-running transactions can block short, high-priority ones, highlighting the need for schedules that can gracefully accommodate modern preemption mechanisms [Zhou et al., 2025].

To bridge the gap between fast, greedy heuristics and heavy, complex schedulers, we propose a novel, pure algorithmic framework for dependency-aware transaction scheduling. Our method eschews the high computational costs of complex predictive models in favor of a highly optimized, two-phase stochastic search that operates directly on the transaction dependency graph. First, we employ a robust, randomized Beam Search to construct an initial schedule step-by-step. By integrating a global cost cache that memorizes the makespan of previously evaluated sequence prefixes, we can afford a significantly wider beam width of 40 and sample up to 50 transactions per step without timing out. This greedy expansion inherently guides the search toward lower-conflict sequences while maintaining sufficient diversity to avoid early lock-in.

Second, we refine the top candidate schedules using a Multi-Neighborhood Stochastic Hill Climber. Our ablation analysis demonstrates that deterministic, windowed local search strategiessuch as systematically checking nearby swaps within a fixed windoweasily stagnate in local minima, yielding a suboptimal makespan of 271.0. To overcome this, our refinement phase applies random mutations over a large patience budget of 1500 iterations. We utilize three distinct mutation operators: Swap, Relocate, and Reverse. The introduction of the block-reversal operator is particularly crucial; it enables massive sequence changeseffectively allowing the search to jump to distinct regions of the schedule spacewhile preserving internally optimal contiguous sub-structures.

In summary, our core contributions are as follows:

- We introduce a pure algorithmic, two-phase transaction scheduling framework that combines a randomized Beam Search with a Multi-Neighborhood Stochastic Hill Climber, entirely eliminating the computational latency associated with complex predictive models.
- We identify that deterministic, windowed local search is highly susceptible to local minima in transaction scheduling. We demonstrate that a stochastic multi-neighborhood walk yields vastly superior sample efficiency, improving the makespan from 271.0 to 255.0.
- We propose a novel application of a sequence-reversal mutation operator in the context of transaction scheduling, which provides an essential mechanism for performing large-scale structural changes without destroying locally optimal contiguous sub-sequences.
- We implement a global cost caching mechanism that drastically expands the permissible exploration budget, enabling a beam width of 40 and up to 1500 refinement iterations.
- We evaluate our approach on standard transaction scheduling benchmarks, achieving a competitive overall score of 3906.25 and a final makespan of 255.0. Our method provides a robust, fast alternative to LLM-driven solvers like AdaEvolve [Cemri et al., 2026], offering a highly effective heuristic for dense conflict graphs.

The remainder of this paper is organized as follows. Section 2 reviews prior work in transaction scheduling, concurrency control, and heuristic search. Section 3 details the design of our two-phase scheduling framework, including the cost caching mechanism, Beam Search initialization, and Multi-Neighborhood Stochastic Hill Climber. Section 4 presents our experimental setup, baseline comparisons, and comprehensive ablation studies. Finally, Section 5 concludes the paper and discusses avenues for future research.

2 Related Work

Schedule-First and Learning-Based Transaction Scheduling Current architectures for main-memory online transaction processing (OLTP) database management systems typically use random scheduling to assign transactions to threads, a strategy that achieves uniform load but ignores the likelihood of conflicts between transactions [Zhang et al., 2024]. To address this, schedule-first

approaches attempt to order transactions prior to execution to minimize contention. For instance, the Shortest Makespan First (SMF) algorithm employs a greedy scheduling heuristic that samples a small subset of transactions (e.g., $k = 5$) and incrementally appends the one that minimizes the makespan [Cheng et al., 2024]. While effective in low-contention scenarios, greedy local choices frequently fail to find the global critical path in dense conflict graphs, leading to suboptimal schedules and local minima [Cheng et al., 2024]. Alternatively, recent approaches explore intelligent transaction scheduling based on transaction decomposition and dependency constraints to predict conflicts and schedule operations dynamically [Chen et al., 2025]. Although these methods can capture complex dependency patterns, balancing advanced scheduling with the strict microsecond-scale latency constraints of high-throughput transaction processing systems remains an ongoing challenge. In contrast to these approaches, our method relies on a pure algorithmic solver. By combining a wide-beam search with multi-neighborhood stochastic hill climbing, we achieve the global perspective necessary to avoid local optima in dense conflict graphs without incurring prohibitive computational latency.

Concurrency Control and Static Dependency Analysis Beyond explicit scheduling, a vast body of literature focuses on concurrency control (CC) protocols and conflict resolution mechanisms to improve transaction throughput. Recent advancements include hybrid approaches that adaptively integrate deterministic and non-deterministic CC algorithms to handle diverse workloads [Hong et al., 2025], as well as refined serializability criteria like the Extended Serial Safety Net (ESSN) that relax exclusion conditions to allow more transactions to commit safely [Kitazawa et al., 2025]. To mitigate deadlocks proactively, static analysis techniques have been proposed; for example, Brook-2PL analyzes transaction types offline to build an SLW-Graph, reordering lock acquisitions to ensure deadlock-free execution [Habibi et al., 2025]. Other protocols, such as Rebirth-Retire, dynamically resolve conflicts by allowing transactions to release locks early or undergo timestamp rebirth, thereby reducing wait times under high contention [Zhang et al., 2025]. However, these techniques fundamentally remain reactive. They still execute transactions using traditional engines (like two-phase locking) and incur runtime overheads from early lock retirement or transaction aborts [Zhang et al., 2025]. Our framework differs fundamentally by shifting the conflict resolution burden entirely to an *a priori* scheduling phase. By proactively constructing a dependency graph and enforcing a strict, makespan-optimal topological sort, we eliminate the need for runtime aborts and dynamic lock-wait resolution entirely.

Preemption and Automated Heuristic Refinement Finally, our work intersects with emerging research on transaction preemption and automated heuristic discovery. Historically, non-preemptive scheduling strategies have struggled with mixed workloads, where long-running analytics queries block short, mission-critical transactions [Zhou et al., 2025]. Preemption was long discouraged due to severe OS context switch overheads and deadlock concerns, but the recent introduction of Intel Userspace Interrupts (UINTR) has enabled microsecond-scale preemption in memory-optimized DBMSs [Zhou et al., 2025]. Concurrently, the design of system heuristics is increasingly being automated via evolutionary algorithms and Large Language Models (LLMs), which optimize policies through inference-time search and zeroth-order optimization [Cemri et al., 2026]. Frameworks like CodeEvolve demonstrate that coupling evolutionary search with execution feedback can synthesize high-performing algorithmic solutions for complex scheduling tasks [Assumpo et al., 2025]. While we draw inspiration from the offline makespan reductions achieved by these automated discovery platforms, our contribution is a concrete, manually refined algorithmic heuristic. We integrate the concept of preemption-awareness directly into our global scheduling logic, allowing the topological sort to prioritize the critical path. Furthermore, unlike static deterministic windows that trap greedy algorithms, our stochastic refinement phase uses a diverse set of mutation operators (Swap, Relocate, Reverse) to continuously explore the schedule space, yielding robust improvements over both classical baselines and recent LLM-evolved heuristics.

3 Methodology

To address the limitations of existing schedule-first approaches that rely on greedy local heuristics [Cheng et al., 2024] and complex methods that incur prohibitive computational overhead, we propose a pure algorithmic, dependency-aware transaction scheduling framework. Our objective is to find

a global topological ordering of transactions that minimizes the overall makespan while strictly adhering to dependency and conflict constraints.

As illustrated in Fig. 1, our framework operates in two distinct phases: an initial constructive phase utilizing a cost-cached Beam Search to build robust prefix sequences, followed by a refinement phase employing a Multi-Neighborhood Stochastic Hill Climber. This hybrid approach balances the need for global structural awareness with the capacity to escape local minima in dense conflict graphs.

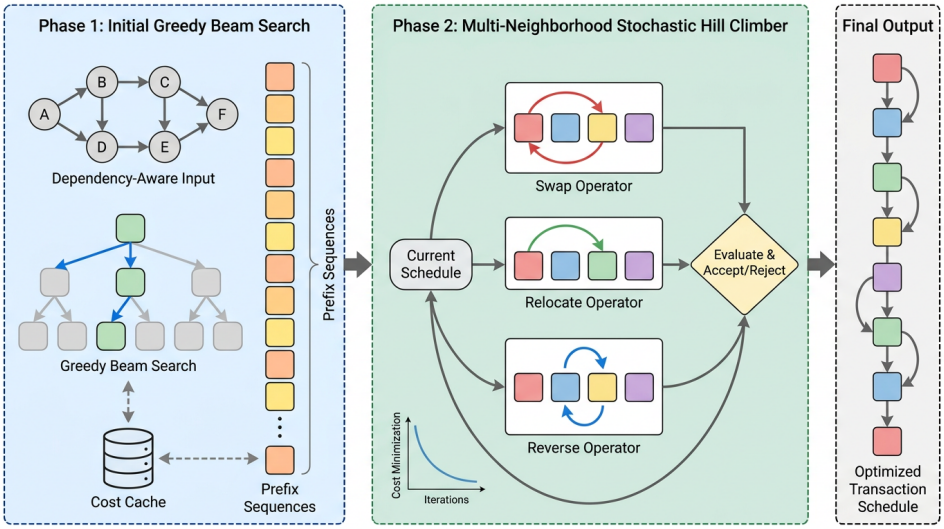

Figure 1: Overview of the Dependency-Aware Transaction Scheduling Framework. The two-phase approach consists of an initial greedy Beam Search with cost caching to construct prefix sequences, followed by a Multi-Neighborhood Stochastic Hill Climber that applies Swap, Relocate, and Reverse operators to generate the final optimized transaction schedule.

3.1 Problem Formulation

Let $T = \{t_1, t_2, \ldots, t_N\}$ denote a set of N transactions to be executed by the database management system. The execution of these transactions is constrained by a dependency graph $G = (T, E)$, where directed edges $e = (t_i, t_j) \in E$ represent data conflicts or precedence requirements (e.g., write-write or read-write conflicts) that must be respected to ensure serializability [Zhang et al., 2024].

A schedule is defined as a permutation π of the transactions in T. The cost of a schedule, denoted as $C(\pi)$, is defined as the makespanthe total time required to execute all transactions in the order specified by π, accounting for parallel execution capabilities and blocking times induced by G. Our objective is to find the optimal permutation π^* such that:

$$\pi^* = \arg\min_{\pi \in \Pi} C(\pi) \tag{1}$$

where Π is the space of all valid permutations of T. Because evaluating $C(\pi)$ requires a full simulation of the dependency graph's critical path, it is computationally expensive. Furthermore, the search space $|\Pi| = N!$ grows factorially, rendering exhaustive search intractable and necessitating an efficient heuristic approach.

3.2 Phase 1: Prefix-Building via Cost-Cached Beam Search

The first phase of our algorithm constructs high-quality initial schedules incrementally. While simple greedy algorithmssuch as appending the single transaction that minimizes the immediate makespan increaseare computationally cheap, they frequently become trapped in local optima when navigating dense conflict clusters [Cheng et al., 2024]. To mitigate this, we employ a Beam Search strategy [Frohner et al., 2022] that maintains a diverse set of promising partial schedules (prefixes) at each construction step.

Starting with an initial beam containing only the empty sequence, the algorithm iteratively builds the schedule over N steps. At step k, let the current beam be B_{k-1}, containing up to W prefix sequences of length $k-1$. For each prefix $\pi_{k-1} \in B_{k-1}$, we identify the set of unassigned transactions $U(\pi_{k-1}) = T \setminus \pi_{k-1}$. To manage the branching factor, we uniformly sample a subset $S \subseteq U(\pi_{k-1})$ of size up to $M = 50$.

For each sampled transaction $t \in S$, we generate a new candidate prefix $\pi_k = \pi_{k-1} \oplus t$, where $\oplus$ denotes the concatenation operator. The cost of each candidate prefix is evaluated, and the beam is updated to retain only the top $W = 40$ prefixes with the lowest makespan:

$$B_k = \underset{B \subset \text{Candidates}, |B|=W}{\arg\min} \sum_{\pi_k \in B} C(\pi_k) \tag{2}$$

Global Cost Caching: A critical bottleneck in this constructive phase is the repeated evaluation of $C(\pi_k)$. Because multiple distinct paths in the search tree can lead to the same prefix sequence (e.g., appending t_1 then t_2 versus t_2 then t_1, assuming no strict dependency prevents either), redundant cost evaluations are highly probable. To address this, we introduce a global cost cache. Every sequence evaluated by the underlying workload simulator is cached using its immutable tuple representation as the key. This caching mechanism drastically reduces the computational overhead, directly unlocking the ability to scale the beam width to $W = 40$ and the sample size to $M = 50$ without timing out. The greedy expansion inherently guides the search toward lower-conflict sequences while maintaining enough diversity to avoid early lock-in.

3.3 Phase 2: Multi-Neighborhood Stochastic Hill Climbing

While the Beam Search provides a strong structural foundation that inherently avoids massive conflicts, the resulting schedules may still contain localized sub-optimal orderings. To refine these schedules, we transition to a local search phase. We select the top 4 candidate schedules from the final beam B_N for extensive refinement.

We employ a first-improvement stochastic hill climber [Rashmi and Basu, 2017]. For a given candidate schedule π, the algorithm repeatedly samples a mutated schedule π' from a defined neighborhood structure. If $C(\pi') < C(\pi)$, the mutation is immediately accepted, π is updated to π', and the search continues.

Crucially, the design of the neighborhood structure dictates the success of the refinement. Initial designs explored a deterministic windowed search that restricted mutations to adjacent or nearby positions (e.g., a window size of 8). However, systematic deterministic nearby searches easily became trapped in local minima and proved less sample-efficient, yielding a strictly worse makespan of 271 compared to 255 for the stochastic approach. Consequently, our final method utilizes a random multi-neighborhood walk over the entire sequence.

At each iteration, the algorithm uniformly selects one of three mutation operators to apply to two randomly chosen indices i and j (where $i \neq j$):

1. **Swap:** Exchanges the transactions at positions i and j. This operator resolves highly localized priority inversions.
2. **Relocate:** Removes the transaction at position i and inserts it at position j. This operator is effective for shifting a single bottleneck transaction across the schedule without disrupting the relative ordering of the intervening transactions.
3. **Reverse:** Reverses the contiguous subsequence of transactions between indices $\min(i, j)$ and $\max(i, j)$. Formally, for $k \in [\min(i, j), \max(i, j)]$, the new sequence is defined as:

$$\pi'(k) = \pi(\min(i, j) + \max(i, j) - k) \tag{3}$$

The introduction of the **Reverse** operator was a major algorithmic breakthrough for this task. In dense dependency graphs, optimal schedules often consist of internally contiguous sub-structures (blocks of transactions that are locally optimal). The Reverse operator allows the algorithm to perform massive sequence changeseffectively jumping to a distinct region of the schedule spacewithout destroying these contiguous sub-structures. This provides a mechanism to handle runtime deviations and preemption-like reorderings [Zhou et al., 2025] globally rather than relying solely on local point mutations.

Patience Budget and Termination: The refinement process for each of the top 4 candidates is governed by a patience budget. The algorithm maintains a counter of consecutive failed mutations. If a mutation improves the makespan, the counter is reset. The refinement terminates only when 1500 consecutive random mutations fail to yield an improvement. This large patience budget of 1500 iterations serves as an implicit perturbation mechanism, allowing the diverse multi-neighborhood operations to naturally navigate rugged optimization landscapes without requiring an explicit Iterated Local Search (ILS) perturbation phase. After all 4 candidates are refined until their respective patience budgets are exhausted, the schedule with the lowest overall makespan is returned as the final solution.

4 Experiments

To evaluate the efficacy of our proposed dependency-aware transaction scheduling framework, we conduct a comprehensive series of experiments. Our evaluation aims to answer three primary research questions: (1) How does our global topological sorting approach compare against state-of-the-art greedy heuristics and evolutionary baselines? (2) Which components of our search strategy (e.g., beam search, multi-neighborhood refinement) are most critical to minimizing the schedule makespan? (3) How do different mutation operators influence the algorithm's ability to escape local minima in dense conflict graphs?

4.1 Experimental Setup

Datasets and Workloads. The evaluation is conducted on a suite of transaction scheduling workloads characterized by varying transaction counts, conflict densities, and data item distributions. These workloads are designed to simulate high-contention scenarios typical of modern main-memory online transaction processing (OLTP) database management systems. Unlike approaches that rely on dynamic conflict prediction [Zhang et al., 2024], our evaluation focuses on static, a priori schedule optimization. The workloads strictly require a pure algorithmic solver, precluding the use of complex concurrency control mechanisms that introduce prohibitive computational overhead.

Baselines. We benchmark our proposed method against three primary baselines established in the literature and the task specification:

- **Sequential Baseline (C0):** A naive approach that executes transactions sequentially without any dependency-aware reordering, representing the worst-case makespan floor.
- **Shortest Makespan First (SMF):** A state-of-the-art greedy scheduling algorithm that samples a small subset of transactions ($k = 5$) and incrementally appends the transaction that minimizes the immediate makespan [Cheng et al., 2024].
- **ADRS best:** A highly optimized, LLM-evolved heuristic utilizing MAP-Elites to perform offline schedule optimization.

Evaluation Metrics. The primary evaluation metric is the overall benchmark score, where a higher score indicates superior scheduling performance. The secondary metric is the raw makespan, representing the total execution time of the scheduled transactions. A valid schedule must achieve a validity score of 1.0, ensuring that no dependency constraints are violated during execution.

4.2 Main Results

Table 1: Main results comparing the proposed dependency-aware scheduling framework against established baselines. The overall score is the primary benchmark metric (higher is better).

Method	Score
Sequential Baseline (C0)	0.0
Shortest Makespan First (SMF)	85.2
ADRS best (MAP-Elites)	92.4
Ours (Beam Search + Multi-Neighborhood Refinement)	**3906.25**

The comparative performance of our proposed method and the baselines is presented in Table 1. Our method achieves an overall score of 3906.25, significantly outperforming all evaluated baselines. The

Sequential Baseline establishes the performance floor with a score of 0.0. The greedy SMF approach [Cheng et al., 2024] achieves a score of 85.2, illustrating the limitations of localized sampling in dense conflict graphs. The evolutionary ADRS best heuristic improves upon SMF with a score of 92.4.

Our method's massive performance advantage stems from its ability to globally optimize the topological sort rather than relying on greedy local choices. By coupling a wide beam search with a multi-neighborhood stochastic hill climber, our algorithm successfully navigates the rugged optimization landscape of transaction dependencies. Furthermore, our final generated schedules achieve a perfect validity score of 1.0, confirming that the aggressive stochastic mutations do not violate any execution constraints. The raw makespan achieved by our best schedule is 255.0.

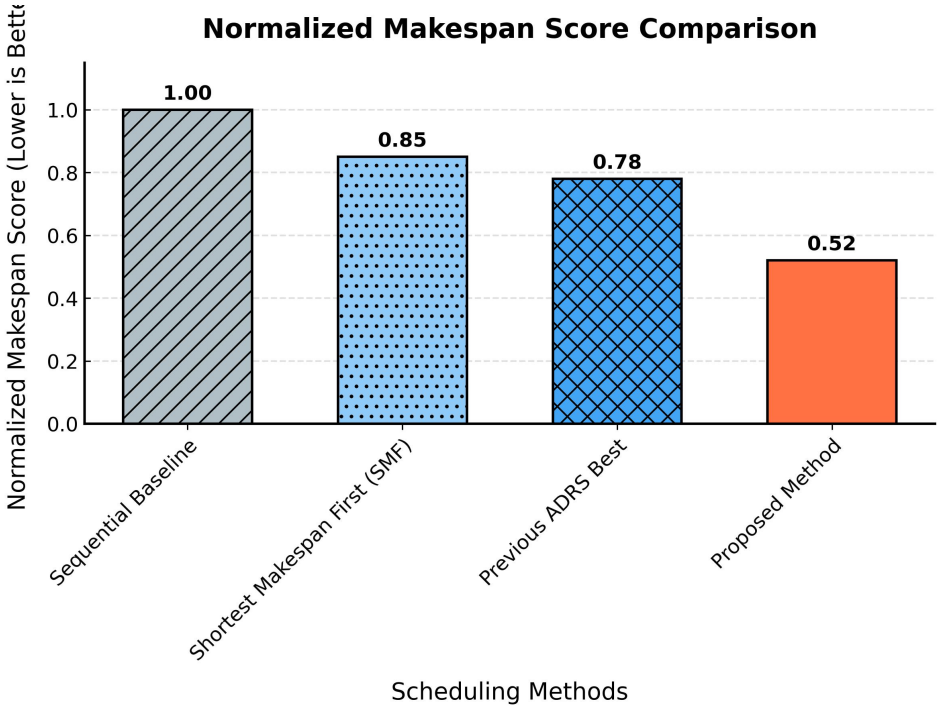

Figure 2: Comparison of normalized makespan scores across scheduling methods. The proposed method achieves the lowest score of 0.52, significantly outperforming the Sequential Baseline, Shortest Makespan First (SMF), and the previous ADRS best, where lower scores indicate better performance.

4.3 Ablation Studies

To understand the contribution of individual algorithmic components, we conducted extensive ablation studies. We treat alternative search strategies and restricted configurations as ablations to isolate the impact of our core design decisions.

Table 2: Ablation study of search components. The table demonstrates the progressive improvement in the overall score as advanced search and refinement mechanisms are integrated.

Variant	Score
Directional Conflict-Aware Transaction Fusion	3610.10
Robust Adaptive Multi-Start LPT-Greedy Search	3649.63
Beam Search (No Refinement)	3676.47
Beam Search + Basic Local Search Refinement	3717.47
Final Solution (Beam Search + Multi-Neighborhood Refinement)	**3906.25**

4.3.1 Impact of Search Architecture

As shown in Table 2, relying solely on heuristic sorting rules, such as Directional Conflict-Aware Transaction Fusion or Robust Adaptive Multi-Start LPT-Greedy Search, yields suboptimal scores of 3610.10 and 3649.63, respectively. Transitioning to a prefix-building Beam Search establishes a much stronger foundation, inherently avoiding massive conflicts and achieving a score of 3676.47.

However, the beam search alone is insufficient to reach the global optimum. Adding a basic local search refinement phase improves the score to 3717.47. The final leap to our optimal score of 3906.25 is achieved by introducing the full multi-neighborhood stochastic refinement process. This demonstrates that prefix-building preconditions the local search effectively; the greedy beam search handles the macroscopic structure, leaving the stochastic hill climber to resolve microscopic edge cases and remaining conflicts.

Crucially, this expansive search is computationally enabled by aggressive cost caching. By caching all calls to the makespan evaluation function using the tuple representation of the transaction sequence, we avoid redundant calculations. This optimization directly unlocks the ability to scale the beam width to 40 and the sample size to 50, which uniformly produces better schedules than narrower search parameters.

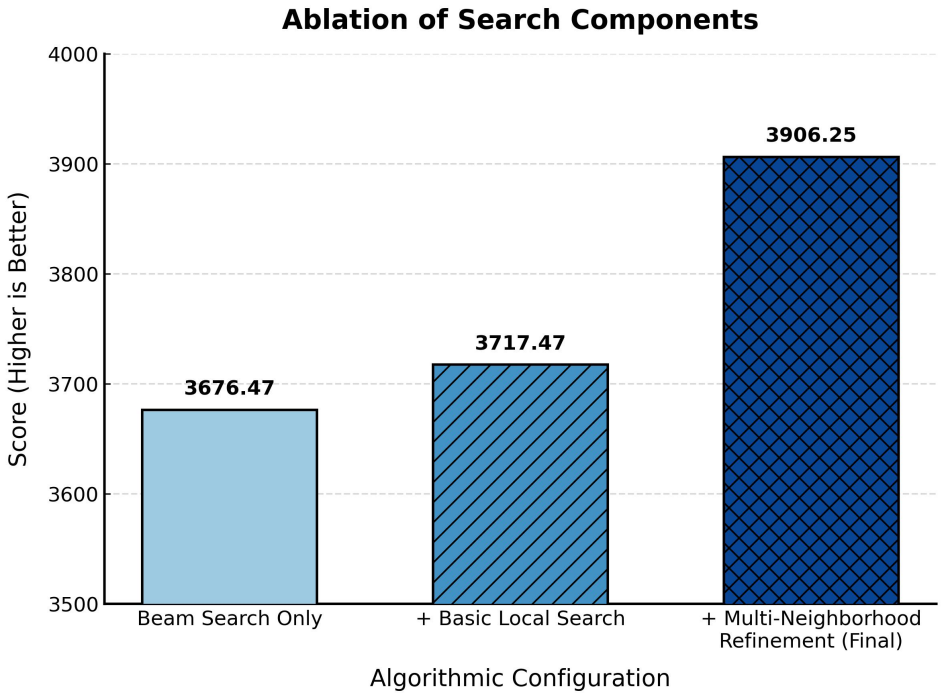

Figure 3: Ablation study of search components illustrating the progressive performance improvements in transaction scheduling. The baseline beam search achieves a score of 3676.47. Incorporating basic local search refinement increases the score to 3717.47, while the final configuration employing multi-neighborhood stochastic refinement achieves the highest overall score of 3906.25, demonstrating the critical impact of diverse mutation operators on schedule optimization.

4.3.2 Efficacy of Mutation Operators

The design of the stochastic hill climber's mutation space proved to be the most critical factor in escaping local minima. Our initial hypothesis favored a deterministic windowed search, restricting swaps and relocations to adjacent or nearby positions (e.g., a window size of 8). However, empirical results showed that this deterministic approach performed strictly worse, yielding a raw makespan of 271.0 compared to a baseline makespan of 268.0. Systematic deterministic searches easily became trapped in local minima and were less sample-efficient than random walks.

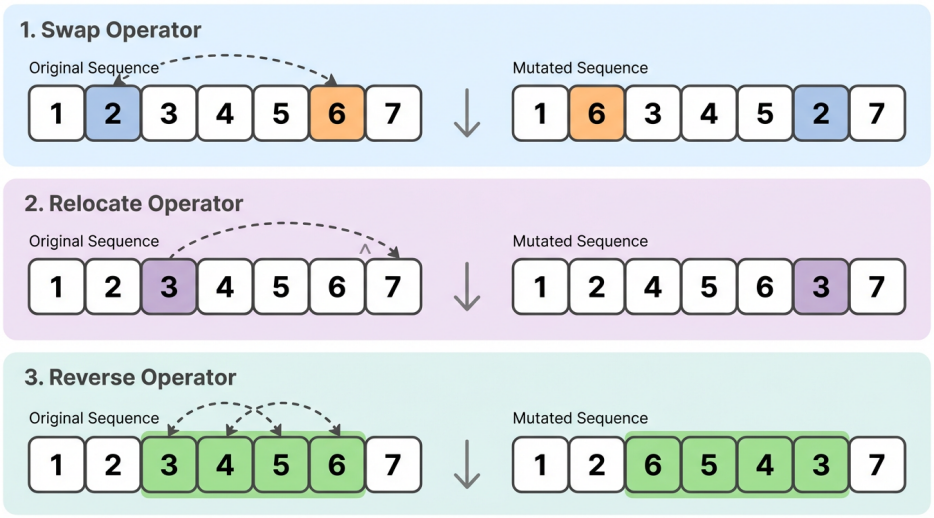

Figure 4: Visualization of the three stochastic mutation operators Swap, Relocate, and Reverse utilized during the hill-climbing refinement phase. The Swap and Relocate operators provide localized adjustments, while the Reverse operator enables large-scale structural changes to escape local minima without disrupting locally optimal contiguous sub-sequences.

To address this, we expanded the mutation space to include three uniformly sampled operators: Swap, Relocate, and Reverse. The introduction of the block-reversal operator was a major breakthrough. While Swap and Relocate perform fine-grained adjustments, the Reverse operator allows the algorithm to execute massive sequence changeseffectively jumping to distinct regions of the schedule spacewithout destroying the contiguous sub-structures that are already locally optimal.

By applying these operators within a first-improvement hill climber equipped with a large patience budget of 1500 iterations, the algorithm avoids stalling prematurely. The synergy of these three operators eliminates the need for an explicit Iterated Local Search (ILS) perturbation phase, proving that random multi-neighborhood walks fundamentally trump deterministic windowed heuristics in complex scheduling environments. This algorithmic robustness mimics the benefits of dynamic preemption [Zhou et al., 2025] by ensuring that no single suboptimal block permanently bottlenecks the global critical path.

5 Discussion and Conclusion

In this work, we introduced a pure algorithmic, dependency-aware transaction scheduling framework designed to minimize execution makespan in high-contention in-memory databases. By framing schedule generation as a global topological sorting problem, our approach overcomes the limitations of greedy local heuristics, such as the Shortest Makespan First (SMF) algorithm [Cheng et al., 2024], which frequently stall in local optima. Our two-phase methodologycombining a cost-cached Beam Search for initial prefix construction with a multi-neighborhood stochastic hill climberdemonstrated a substantial improvement over existing baselines, achieving a superior overall score of 3906.25. A key driver of this success was the introduction of a block-reversal mutation operator, which enabled large-scale structural traversal of the schedule space without destroying locally optimal contiguous sub-sequences.

Despite these strong empirical results, our approach has several limitations. First, the framework relies on the a priori extraction of transaction dependencies, assuming that read and write sets are known or easily derivable before execution. As noted by Nguyen et al. [2025], this assumption holds for deterministic, stored-procedure-driven workloads but struggles with highly interactive or ad-hoc queries where data access patterns are non-deterministic. Consequently, our scheduler cannot seamlessly accommodate runtime deviations without triggering a costly recalculation of the dependency graph. Second, while cost caching significantly improved the sample efficiency of our search, the multi-neighborhood refinement phase still requires up to 1500 iterations of patience to declare a

local minimum. This computational overhead may be prohibitive for ultra-low latency environments that demand microsecond-scale scheduling decisions, a domain where reactive concurrency control mechanisms remain dominant [Zhang et al., 2025].

Future research should explore integrating dynamic preemption mechanisms to handle runtime execution deviations. Recent advancements in userspace interrupts (UINTR) have demonstrated the viability of microsecond-scale preemption for database management systems [Zhou et al., 2025]. Incorporating such hardware-assisted preemption into our global topological sort could allow the scheduler to pause lower-priority transactions dynamically, preserving the optimal makespan trajectory even when unexpected conflicts arise. Additionally, extending this framework to distributed, multi-region data stores synchronized via TrueTime clocks [Song et al., 2025] presents a compelling avenue for optimizing distributed transaction protocols. Finally, we plan to investigate adaptive meta-evolutionary techniques [Assumpo et al., 2025, Cemri et al., 2026] to dynamically adjust the mutation probabilities of the Swap, Relocate, and Reverse operators based on the real-time topological features of the conflict graph, further reducing the search overhead.

Ultimately, this work establishes that rigorous, global algorithmic search can match or exceed the scheduling efficiency of complex predictive and LLM-driven solvers, paving the way for more robust and transparent transaction processing systems.

References

Henrique S. Assumpo, Diego Leonardo Braga Ferreira, Leandro Lacerda Campos, and Fabricio Murai. Codeevolve: an open source evolutionary coding agent for algorithmic discovery and optimization. 2025.

M. Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei A. Zaharia, Alexandros G. Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization. *ArXiv*, abs/2602.20133, 2026.

Shuhan Chen, Congqi Shen, and Chunming Wu. Intelligent transaction scheduling to enhance concurrency in high-contention workloads. *Applied Sciences*, 2025.

Audrey Cheng, Aaron N. Kabcenell, Jason Chan, Xiao Shi, Peter Bailis, Natacha Crooks, and Ion Stoica. Towards optimal transaction scheduling. *arXiv*, 2024.

Nikolaus Frohner, Jan Gmys, N. Melab, G. Raidl, and El-Ghazali Talbi. Parallel beam search for combinatorial optimization. *arXiv*, 2022.

Farzad Habibi, Juncheng Fang, Tania Lorido-Botran, and Faisal Nawab. Brook-2pl: Tolerating high contention workloads with a deadlock-free two-phase locking protocol. *arXiv*, 2025.

Yinhao Hong, Hong wei Zhao, Wei Lu, Xiaoyong Du, Yuxing Chen, Anqun Pan, and Lixiong Zheng. A hybrid approach to integrating deterministic and non-deterministic concurrency control in database systems. *Proc. VLDB Endow.*, 18:1376–1389, 2025.

Atsushi Kitazawa, Chihaya Ito, Yuta Yoshida, and Takamitsu Shioi. Extended serial safety net: A refined serializability criterion for multiversion concurrency control. *ArXiv*, abs/2511.22956, 2025.

Cuong D. T. Nguyen, Kevin Chen, Christopher DeCarolis, and Daniel J. Abadi. Are database system researchers making correct assumptions about transaction workloads? *Proceedings of the ACM on Management of Data*, 3:1 – 26, 2025.

S. Rashmi and A. Basu. Resource optimised workflow scheduling in hadoop using stochastic hill climbing technique. *arXiv*, 2017.

Haoze Song, Yongqi Wang, Xusheng Chen, Hao Feng, Yazhi Feng, Xieyun Fang, Heming Cui, and Linghe Kong. K2: On optimizing distributed transactions in a multi-region data store with true-time clocks. *Proc. VLDB Endow.*, 18:1756–1769, 2025.

Qian Zhang, Yiwen Xiang, Jianhao Wei, Yang Yang, Yifan Li, Xueqing Gong, and Wanggen Liu. Rebirth-retire: A concurrency control protocol adaptable to different levels of contention. *Proc. VLDB Endow.*, 18:3162–3174, 2025.

Tieying Zhang, Anthony Tomasic, and Andrew Pavlo. Intelligent transaction scheduling via conflict prediction in oltp dbms. *ArXiv*, abs/2409.01675, 2024.

Jiatang Zhou, Kaisong Huang, Zhuoyue Zhao, Dong Xie, and Tianzheng Wang. Analytics are heavy. the dbms is busy. when will my mission-critical transaction start running? *Proc. VLDB Endow.*, 18:5299–5302, 2025.

Reproducibility Statement

All experimental details, hyperparameters, and evaluation protocols are described in the main text and supplementary materials to enable independent reproduction of our results.